```
/* Babel service worker — offline app
shell.
   Bump CACHE on every release so clients
pull the new files. */
const CACHE = 'babel-v79';
const ASSETS = [
  'Babel.html',
  'index.html',
  'manifest.webmanifest',
  'icons/babel-192.png',
  'icons/babel-512.png',
  'icons/babel-maskable-512.png'
];

self.addEventListener('install', (e) => {
  e.waitUntil(
    caches.open(CACHE).then((c) =>
c.addAll(ASSETS)).then(() =>
self.skipWaiting())
  );
});

self.addEventListener('activate', (e) => {
  e.waitUntil(
    caches.keys()
      .then((keys) =>
Promise.all(keys.filter((k) => k !==
CACHE).map((k) => caches.delete(k))))
      .then(() => self.clients.claim())
  );
```

```
});

// Let the page trigger an immediate
activation of a waiting worker.
self.addEventListener('message', (e) => {
  if (e.data === 'SKIP_WAITING')
self.skipWaiting();
});

// Navigations: network-first (so a freshly
hosted version shows up), fall back
// to cache when offline. Other assets:
cache-first with background refresh.
self.addEventListener('fetch', (e) => {
  const req = e.request;
  if (req.method !== 'GET') return;
  // Only handle same-origin requests — let
cross-origin (Supabase, CDN) pass straight
through.
  if (new URL(req.url).origin !==
self.location.origin) return;
  const isNav = req.mode === 'navigate' ||
(req.headers.get('accept') ||
'').includes('text/html');
  if (isNav) {
    e.respondWith(
      fetch(req)
        .then((res) => { if (res && res.ok)
{ const copy = res.clone();
caches.open(CACHE).then((c) => c.put(req,
copy)); } return res; })
        .catch(() =>
caches.match(req).then((r) => r ||
caches.match('Babel.html')))
```

```
    );
    return;
  }
  e.respondWith(
    caches.match(req).then((cached) => {
      const net = fetch(req).then((res) =>
{
        if (res && res.ok) { const copy =
res.clone(); caches.open(CACHE).then((c) =>
c.put(req, copy)); }
        return res;
      }).catch(() => cached ||
Response.error());
      return cached || net;
    })
  );
});
```